Amazon Book Rating Network Modeling and Analysis

Fahim Ahmed
Department of Electrical and Computer Engineering
University of Central Florida
Orlando, FL, USA
fahim.ahmed@ucf.edu

Jenna Goodrich
Department of Electrical and Computer Engineering
University of Central Florida
Orlando, FL, USA
jenna.goodrich@ucf.edu

***Abstract*—Link prediction is a fundamental problem in network analysis, aimed at estimating the likelihood of future or missing connections between nodes in a graph. This project explores the use of machine learning techniques for link prediction on a real-world dataset. The dataset, sourced from Kaggle and titled Amazon Books Reviews, represents a bipartite graph where nodes correspond to users and books, and edges represent review scores. This forms a social network-style interaction graph based on user-book relationships. After conducting exploratory data analysis, we applied both traditional link prediction methods and a graph convolutional network (GCN). Among the heuristics tested, the Jaccard Coefficient, Preferential Attachment, Adamic-Adar Index, Resource Allocation Index, and Katz Centrality, Katz achieved the highest performance and outperformed our simple GCN model, measured by AUROC and AUPR scores.**

***Index Terms*—network analysis, recommendation system, link prediction, bipartite graph, graph convolution network**

I. Introduction

Recommendation systems have become increasingly important for various use areas including e-commerce, entertainment, and social media. Netflix, Amazon, YouTube, LinkedIn, and more use recommendation systems to suggest products, people, and other content to users.

The main goal of this project was to apply machine learning tools and algorithms to a specific task for a real-world dataset. The task focused on in this project is link prediction. We apply various algorithms to get a baseline for comparison against a graph convolutional network (GCN).

The dataset we worked with is the Amazon Books Reviews dataset, which can be modeled as a bipartite graph where one set of nodes represents users and the other represents books. Edges between these nodes represent user-book interactions in the form of review scores. Bipartite graphs introduce unique challenges for link prediction, particularly because many standard heuristics rely on the concept of shared neighbors, which do not exist between nodes of disjoint sets.

In this project, we implemented and compared both traditional heuristic-based link prediction methods and a learning-based approach using a Graph Convolutional Network. Our goal was to evaluate how well a simple GCN can learn to predict user-book links in a sparse, featureless network, and how its performance compares to that of established graph algorithms. We also discuss limitations of both approaches and identify potential directions for future improvement, including feature engineering and model tuning.

II. Related Work

Bipartite networks are represent systems involving interactions between two entities, that is, a bipartite graph is a graph whose nodes can be divided into two disjoint and independent sets, in this case users and books, U and B such that for all edges $e \in E, e = (u, b), u \in U, b \in B$. Compared to standard, unipartite networks, the bipartite network structure requires a different approach for link prediction [1]. Link prediction is a fundamental task in network analysis that involves predicting the likelihood of future or missing connections between nodes in a graph. Traditional heuristic-based methods rely on various similarity scores, but often struggle with scalability especially in sparse networks. The following section outlines the traditional methods used in this work.

A. Link Prediction with Heuristic-Based Methods

Several link prediction algorithms that rely on

1) Jaccard Coefficient: The Jaccard coefficient of two nodes u and v is defined as

$$JC_{(u,v)} = \frac{|\Gamma(u) \bigcap \Gamma(v)|}{|\Gamma(u) \bigcup \Gamma(v)|} \tag{1}$$

where $\Gamma(u)$ denotes the set of neighbors of u. This similarity metric measures the probability that both of two nodes u and v have a feature that either u or v has, in this case that feature is neighbors [2].

2) Adamic-Adar Index: The Adamic-Adar Index is similar to Jaccard coefficient but refines the counting of common neighbors by assigning weights [3] and is defined as

$$AA_{uv} = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{\log |\Gamma(w)|} \tag{2}$$

3) Resource Allocation Index: Resource allocation index measures resource transfer over a network utilizing common neighbors once again and is defined as

$$RA_{(u,v)} = \sum_{w \in \Gamma(u) \cap \Gamma(v)} \frac{1}{|\Gamma(w)|} \tag{3}$$

4) Preferential Attachment: Preferential attachment assumes links are more likely between nodes with higher degrees and is defined as [1].

$$PA_{(u,v)} = k_u \times k_v \tag{4}$$

5) Katz Centrality/Index: Katz centrality measures the connectivity between nodes by considering the number of paths penalized with an attenuation factor α [1] and in the NetworkX implementation is defined as

$$x_u = \alpha \sum_j A_{ij} x_j + \beta, \tag{5}$$

It is important to note that methods based on common neighbors of two vertices will not be able to predict well in bipartite graphs, since two vertices that would be connected do not have any common neighbors [4].

B. Link Prediction with Graph Neural Networks and Graph Convolution Networks

Graph Neural Networks (GNNs) have emerged as powerful tools for learning on graph-structured data and have been successfully applied to tasks such as node classification, link prediction, and graph generation. GNNs operate by iteratively aggregating and transforming information from a node's neighbors to learn expressive node embeddings that capture both local and global graph structure [5].

One commonly used architecture is the Graph Convolutional Network (GCN), which updates each node's representation by aggregating normalized feature information from its immediate neighbors. The propagation rule for a standard GCN layer is defined as:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right) \tag{6}$$

where $\hat{A}$ is the adjacency matrix with self-loops, $\hat{D}$ is the corresponding degree matrix, $H^{(l)}$ represents the node features at layer l, $W^{(l)}$ is a learnable weight matrix, and σ is a non-linear activation function such as ReLU.

In the context of link prediction, GCNs are used to generate node embeddings, which are then fed into a scoring function (e.g., dot product or MLP) to estimate the probability of a link between two nodes. In our implementation, we constructed a two-layer GCN model where each node was initialized with a 64-dimensional random embedding. The node embeddings were refined using GCN layers with 128 hidden units, including dropout for regularization.

Positive training samples corresponded to observed user-book interactions, while negative samples were generated by randomly selecting unconnected user-book pairs. The model was trained using the Adam optimizer and binary cross-entropy loss. Model performance was assessed using AUROC and Average Precision (AP) metrics on a held-out test set.

TABLE I
BOOK RATING NETWORK CHARACTERISTICS

Number of Book Nodes	206711
Number of User Nodes	1008961
Total Nodes	1215672
Total Edges	2115811
Density	1.01447×10^{-5}
Average Degree of Book Nodes	10.2356
Average Degree of User Nodes	2.0970

III. METHODOLOGY

The practical network system modeled in this project is a type of social network, and a bipartite network, consisting of book reviews and reviewers, with links representing a review score. The primary task in this project is link prediction, which involves predicting future connections between nodes in a graph, in this case only connections between books and users. This task is a binary classification task where the goal is to predict if a user would read a book. The algorithms used and evaluated for link prediction in our book rating graph include the following heuristic methods which were outlined in Section 2: Jaccard coefficient, preferential attachment, Adamic-Adar index, resource allocation index, and Katz centrality.

IV. IMPLEMENTATION

A. Data

The data used for this project was from a Kaggle dataset titled Amazon Books Reviews [6]. This dataset consists of a book ratings file and a book details file, but only the book ratings file was used in this project. In the original ratings csv file, the columns included book ID, book title, book price, user ID, profile name, review helpfulness, review score, review time, review summary, and review text. Only user ID, book title, and review score were used in the construction of the graph.

A small section of the network is modeled in Figure 1 to illustrate the structure of the graph. Table 1 provides a numerical view of structural characteristics. There are a few things to note about this network. First, the graph is extremely sparse meaning a tiny fraction of all the possible user-book pairs are connected. This is common in recommendation networks as most users interact with just a few items. However, user activity is relatively low even for a recommendation network with users rating a little over 2 books apiece on average. Books have a slightly higher average degree of approximately 10.

B. Setup

The network is divided up into training and testing sets with an 80/20 split. This is done by randomly splitting existing user-book edges into training and test sets.

Negative samples were generated by randomly selecting user-book pairs that do not have an edge in the original dataset. These were used alongside positive edges to train a binary classifier or evaluate ranking metrics.

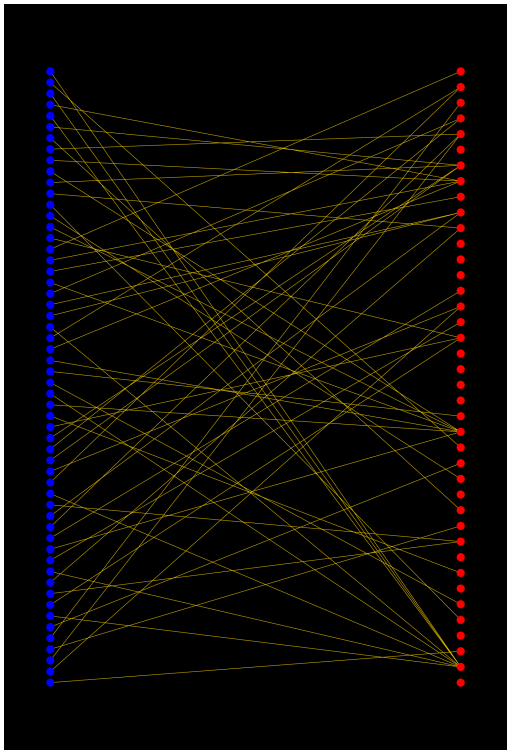

Fig. 1. A sample of the book rating network with blue book nodes b = 80 and red user nodes u = 40

For the heuristic-based methods, we used the NetworkX library to compute similarity scores for test edges using five commonly used measures: Katz Centrality, Preferential Attachment, Jaccard Coefficient, Adamic-Adar Index, and Resource Allocation Index. These scores were evaluated using AUROC and Average Precision (AP) by comparing test edges to randomly sampled negative edges.

The model used for link prediction is a two-layer Graph Convolutional Network (GCN) implemented in PyTorch Geometric. Each node in the graph is initialized with a 64-dimensional learnable embedding. The model consists of two GCN layers with 128 hidden units each and a ReLU activation in between. Dropout with a rate of 0.3 is applied after the first layer to prevent overfitting. The output edge score is calculated using a dot product between the embeddings of node pairs.

Training is performed using the binary cross-entropy loss function and the Adam optimizer with a learning rate of 0.01. The model is trained for 100 epochs, with fixed negative samples across all epochs to ensure consistency. Evaluation is done on a held-out test set using Area Under the ROC Curve (AUROC) and Average Precision (AP). These metrics, along with training loss, are logged at each epoch to assess convergence and performance. Although training continued for all 100 epochs, no early stopping or model checkpointing was applied, which may have contributed to overfitting and a decline in test performance in later epochs.

C. Evaluation

Link prediction is evaluated using two commonly used metrics: Area Under the Receiver Operating Characteristic Curve (AUROC) and Average Precision (AP), also known as Area Under the Precision-Recall Curve (AUPR). AUROC plots the true positive rate against the false positive rate and computes the area under the ROC curve, thereby measuring the ability of a classifier to distinguish between positive and negative links. A score of 1 would mean a model perfectly separated the two classes, while a score of 0.5 equates to random guessing, and anything lower indicates that the model is inversely predicting links.

AUPR provides a measure of the model's efficacy in identifying positive samples and is particularly useful in tasks with class imbalances, where positive examples are much rarer than negative ones [7].

For the heuristic-based methods, AUROC and AP were calculated once by scoring test edges and comparing against randomly sampled non-edges. For the GCN model, AUROC and AP were computed at the end of each training epoch, allowing us to track model performance dynamically over time. This distinction allows a direct comparison of the best achievable scores between heuristics and the GCN model.

V. Results

Firstly, the performance of our heuristic-based algorithms is shown in Table II. Unsurprisingly, the metrics which rely on common neighbors—Jaccard Coefficient, Adamic-Adar Index, and Resource Allocation Index—are unable to predict links in this bipartite network, scoring no better than random guessing.

Preferential Attachment performed second best, with an AUROC score of 0.6660, indicating a moderate ability to distinguish between positive and negative links. Its AP score of 0.7949 also suggests that it performs reasonably well at ranking positive samples, though not as well as Katz Centrality. Katz Centrality had the highest scores for both AUROC and AP, demonstrating its effectiveness at predicting links even in a sparse bipartite graph.

TABLE II
PERFORMANCE EVALUATION OF HEURISTIC-BASED LINK PREDICTION METHODS

Method	AUROC	AP
Katz Centrality	**0.8634**	**0.8921**
Preferential Attachment	0.6660	0.7949
Jaccard Coefficient	0.5000	0.5000
Adamic-Adar Index	0.5000	0.5000
Resource Allocation Index	0.5000	0.5000

To compare with a learning-based approach, we trained a Graph Convolutional Network (GCN) model for 100 epochs. Training loss, AUROC, and Average Precision (AP) were recorded throughout training and are shown in Figures 2 and 3.

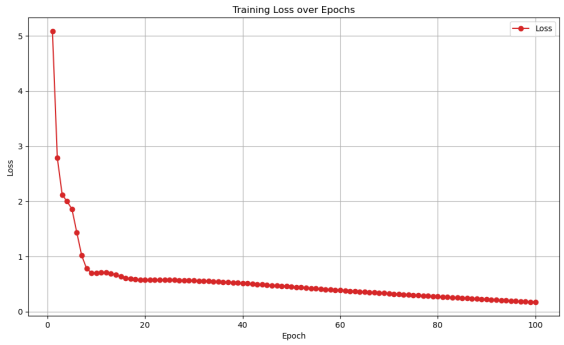

Fig. 2. Training loss over 100 epochs

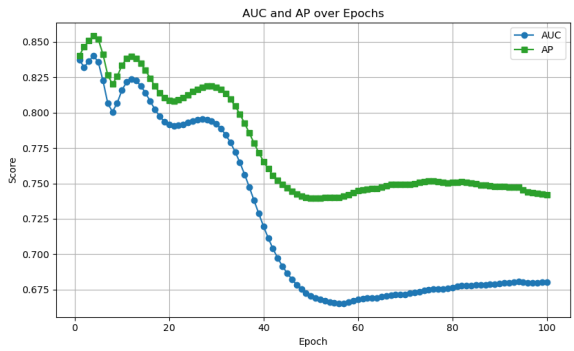

Fig. 3. AUROC and AP scores over 100 epochs

The loss curve in Figure 2 shows a steady decline, indicating successful convergence of the model. However, as shown in Figure 3, both AUROC and AP peaked early—around epochs 5 to 10—reaching values near 0.85, but then experienced a gradual decline as training continued. After epoch 40, the model's performance appeared to stabilize, with AUROC settling around 0.675 and AP around 0.74.

This performance, while significantly better than random guessing (0.5), fell short of Katz Centrality. The degradation in GCN performance after early epochs suggests potential overfitting to noisy training signals, especially since node features were not available and the graph is extremely sparse. These results imply that while GCNs are capable of learning from structural data alone, strong heuristic measures like Katz remain more robust in featureless, imbalanced bipartite graphs.

VI. Conclusion

In this project, we performed preliminary data analysis and implemented various link prediction algorithms as well as a GNN for link prediction. Our work highlights the importance of algorithm selection for bipartite graphs, as many commonly used heuristic-based algorithms do not perform well due to the structure of the graph and a lack of common neighbors between user-book pairs. Of the tested heuristic-based methods, Katz centrality performed best. Additionally, this work provided insight into working with real-world networks that are large and sparse. Another key takeaway is that negative sampling is crucial in order to address the class imbalance between positive and negative edges.

Some of the limitations of this work include the fact that the cold start problem is not explicitly addressed. During training, only nodes in the training edge set are involved in message passing, so nodes with no connections in the training edges don't benefit from updates to the model. This could be mitigated by using features like book metadata or pretraining embeddings. Another limitation is that the GCN used in this work was not optimized and no hyperparamter tuning was performed, which may have improved its performance. More specifically, the model only relies on randomly initialized embeddings rather than any node features. Additionally, the GCN only has two layers which may have been insufficient for capturing long-range dependencies in the way Katz does. In the context of a true real world system, the GCN would be limited as it cannot generalize to new nodes without retraining, but within this work the graph was static. A true recommendation system would be evolving over time and the current setup would be unable to address temporal dynamics. Since we only do link prediction rather than link regression, we are only predicting whether a node exists or not, ignoring the edge weights that represent the rating given by a user. A better implementation would have used only edges of 3 or higher as positive links, though this would make our incredibly sparse graph even more sparse.

This work has several future directions that could be pursued. Other methods of link prediction could be used and evaluated, including more traditional link scoring methods that can handle a bipartite graph structure, embedding-based methods like DeepWalk, and other GNN models with tuned hyperparameters. Additional feature engineering could be done using the book details file from the original dataset to create features based around genre and author which may help with link prediction.

References

[1] D. Özer, G. K. Orman, and V. Labatut, "Link prediction in bipartite networks," *Procedia Computer Science*, vol. 246, p. 2264–2273, 2024, arXiv:2406.06658 [cs].

[2] D. Liben-Nowell and J. Kleinberg, "The link prediction problem for social networks," in *Proceedings of the Twelfth International Conference on Information and Knowledge Management*, ser. CIKM '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 556–559. [Online]. Available: https://doi.org/10.1145/956863.956972

[3] T. Zhou, L. Lu, and Y.-C. Zhang, "Predicting missing links via local information," *The European Physical Journal B*, vol. 71, no. 4, p. 623–630, Oct. 2009, arXiv:0901.0553 [physics].

[4] J. Kunegis, E. W. D. Luca, and S. Albayrak, "The link prediction problem in bipartite networks," no. arXiv:1006.5367, Jun. 2010, arXiv:1006.5367 [cs]. [Online]. Available: http://arxiv.org/abs/1006.5367

[5] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[6] Bekheet, "Amazon books reviews." [Online]. Available: https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews

[7] L. Brown and S. Nel, "A comprehensive overview and evaluation of link prediction techniques," *South African Journal of Industrial Engineering*, vol. 34, no. 3, 2023. [Online]. Available: https://sajie.journals.ac.za/pub/article/view/2960